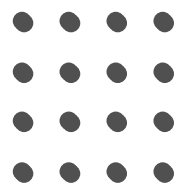




VMES Grinds





?

347. Top K Frequent Elements



347. Top K Frequent Elements

Medium

🔖 Topics

🏢 Companies

Given an integer array `nums` and an integer `k`, return *the `k` most frequent elements*. You may return the answer in **any order**.



Example 1:

Input: `nums = [1,1,1,2,2,3], k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1], k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2], k = 2`

Output: `[1,2]`



Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- k is in the range $[1, \text{the number of unique elements in the array}]$.
- It is **guaranteed** that the answer is **unique**.



Approach 1: Min Heap

Approach 1: Min Heap

- **Count:** Traverse the array once to count the frequency of every number using a Hash Map.
- **Heapify:** Add items to a Min Heap. Keep the size of the heap equal to k by removing the smallest count.
- **Extract:** The elements remaining in the heap are the top k most frequent.



```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) → List[int]:
        count = {}
        for num in nums:
            count[num] = count.get(num, 0) + 1

        minHeap = []
        for n, c in count.items():
            heapq.heappush(minHeap, [c, n])

            if len(minHeap) > k:
                heapq.heappop(minHeap)

        res = []
        for i in range(k):
            res.append(heapq.heappop(minHeap)[1])
        return res
```




Bucket Sort



What is Bucket Sort?

Normally, sorting takes $O(N \log N)$. But here, we know the maximum frequency cannot exceed N (the length of the array).

This allows us to use Bucket Sort:

1. Create an array of buckets where the Index represents the Frequency.
2. Place numbers into their corresponding frequency index.
3. Iterate backwards (from highest index) to gather the top k elements.



Bucket Sort Explained

Step 1 of 12

What is Bucket Sort?

Bucket sort is a distribution-based sorting algorithm that divides elements into buckets and then sorts each bucket individually.

Instead of comparing elements directly, bucket sort **distributes** them into groups (buckets) based on their value range.

Bucket Sort Algorithm Visualization - Interactive Tutorial

Learn bucket sort with step-by-step visual examples. Interactive slideshow covers algorithm steps, time complexity $O(n+k)$, and real coding applications.



Time and Space Complexity Analysis



```
class Solution:
    def topKFrequent(self, nums: List[int], k: int) → List[int]:
        count = {}
        for num in nums:
            count[num] = count.get(num, 0) + 1

        minHeap = []
        for n, c in count.items():
            heapq.heappush(minHeap, [c, n])

            if len(minHeap) > k:
                heapq.heappop(minHeap)

        res = []
        for i in range(k):
            res.append(heapq.heappop(minHeap)[1])
        return res
```

Space - $O(n)$

Time - $O(n \log k)$



```
class Solution:
    def topKFrequent(self, nums, k):
        count = {}
        for num in nums:
            count[num] = count.get(num, 0) + 1

        buckets = [[] for _ in range(len(nums) + 1)]
        for num, freq in count.items():
            buckets[freq].append(num)

        result = []
        for i in range(len(buckets) - 1, 0, -1):
            for num in buckets[i]:
                result.append(num)
                if len(result) == k:
                    return result
        return result
```

Space - $O(n)$

Time - $O(n)$



Key Takeaway

VMES Grinds MMM
Chee sar



- Bucket sort is theoretically faster because of $O(n)$ but has more overhead
- If k is very small (like $k=1$ or $k=5$) and n is huge (like $n=1,000,000$), then $O(n \log k) \approx O(n)$ in practice
- Heap solution is appropriate for this kind of problem because of the word "Top K"